CPSC 254 Assignment #2: Machine Learning Exercise

Report

Team Members and Contribution

Team member	% Contribution
Isaac	100%

Code repository: https://github.com/22mbach/CPSC254-Assignment2

Summary

Part	Topic	Program file	Status
1	Linear regression	linear_reg.py	Complete
2	MLP classification	mlp_classifier.py	Complete
3	K-means clustering	kmeans_clustering.py	Complete
4.1	Gridworld policy iteration	gridworld_policy_iteration.py	Complete
4.2	Snake agent with a DQN	snake-ai-pytorch/agent.py	Complete

All screenshots for Parts 1 to 3 come from running the programs as submitted (Python 3, scikit-learn, matplotlib). A fixed random seed is used, so the numbers below are reproducible.

Part 1: Supervised Learning for Linear Regression (linear_reg.py)

Training dataset, scatter plot and model (steps 1 to 4)

The program loads the 11-point training set, draws a scatter plot with matplotlib, and fits it with scikit-learn's LinearRegression(). A straight line fits the data poorly because the points are U-shaped (they fall from x = -3 to about x = 0 and rise again), so the program also fits a degree-2 polynomial by expanding x into [1, x, x²] with PolynomialFeatures and running the same LinearRegression on those features. The model is still linear in its weights, which is why linear regression can learn a polynomial.

Model	Coefficient vector [w0, w1, …]	Training RMSE
Straight line	[11.4362, 0.9936]	4.7477
Degree-2 polynomial	[4.7878, 1.2333, 1.6910]	0.6348

Learned function (final model): **$y = 4.7878 + 1.2333\,x + 1.6910\,x^2$**

Training error (step 5)

The training error is the root mean squared error, RMSE = $\sqrt{(\ \Sigma(y_i - \hat{y}_i)^2 / N\ )}$, with N = 11. The quadratic model's RMSE of 0.635 is about 7.5 times lower than the straight line's 4.748, so the degree-2 polynomial is the function we keep.

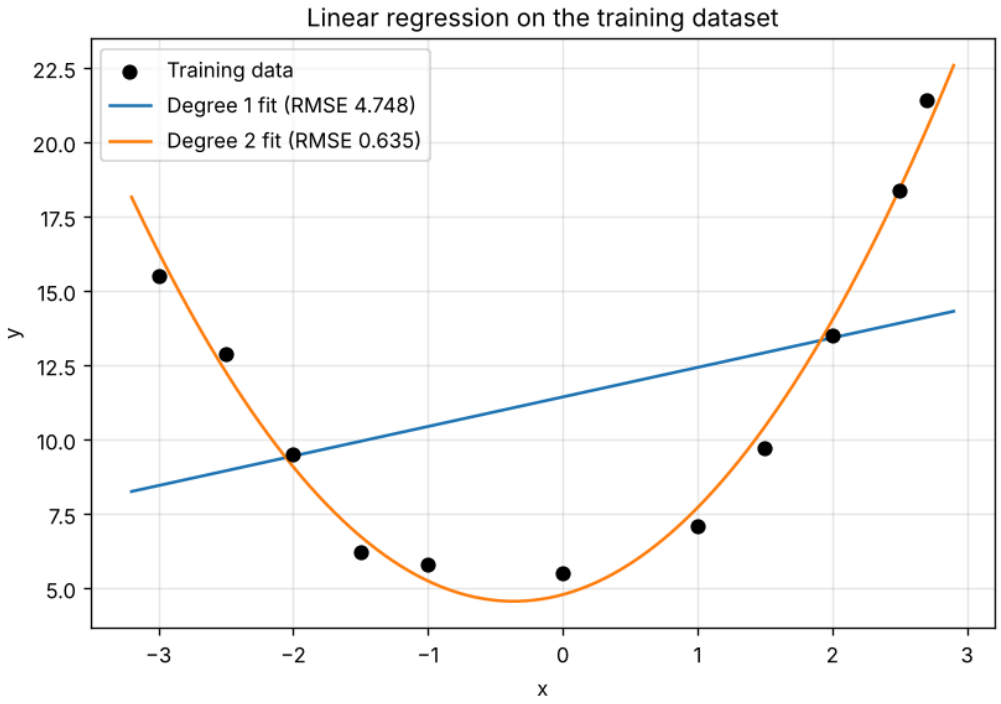


Figure 1. Scatter plot of the training data with the straight-line and degree-2 fits.

```
$ python linear_reg.py
=== (1) Training dataset ===
X = [-3.0, -2.5, -2.0, -1.5, -1.0, 0.0, 1.0, 1.5, 2.0, 2.5, 2.7]
Y = [15.5, 12.9, 9.5, 6.2, 5.8, 5.5, 7.1, 9.7, 13.5, 18.4, 21.4]
N = 11

=== (3)-(5) LinearRegression, straight line ===
Coefficient vector [w0, w1] = [11.4362, 0.9936]
Equation: y = 11.4362 + 0.9936 x
Training RMSE = 4.7477

=== (3)-(5) LinearRegression, polynomial of degree 2 ===
Coefficient vector [w0, w1, w2] = [4.7878, 1.2333, 1.691]
Equation: y = 4.7878 + 1.2333 x + 1.6910 x^2
Training RMSE = 0.6348

The data is U-shaped, so the degree-2 polynomial fits best (training RMSE 0.6348 vs. 4.7477 for a straight line).
Final model: y = 4.7878 + 1.2333 x + 1.6910 x^2

Plot saved to linear_reg.png
```

Figure 2. Terminal output for steps 1 to 5.

Least squares on GasProperties.csv (step 6)

GasProperties.csv has 420,000 rows and five numeric columns (T, P, TC, SV, Idx). We use T, P, TC, and SV as the features and Idx as the target, split the data 80% / 20% (336,000 training rows, 84,000 testing rows), add a column of ones for the bias, and solve the normal equation $\hat{w} = (X^TX)^{-1}X^Ty$ with NumPy (matrix transpose, inverse, and multiplication). This $\hat{w}$ is the closed-form minimizer of $J(w) = (1/N)\ \Sigma(y_i - w^Tx_i)^2$.

Learned function: **Idx = 67.9449 + 0.037207 T − 0.004220 P − 5.990956 TC − 0.059081 SV**

Error	RMSE
Training RMSE	0.136523
Testing RMSE	0.135499

The training and testing RMSE are almost identical, so the model generalizes well and is not overfitting. The error is about half of the standard deviation of Idx (0.2518), and R^2 on the test set is 0.71; the remaining error comes from the relationship between the gas properties and Idx not being fully linear.

```
$ python linear_reg.py   (continued: Part 1.6)
=== (6) Least squares on GasProperties.csv ===
Loaded 420000 rows, columns: T, P, TC, SV, Idx
Features: T, P, TC, SV  ->  target: Idx
Training set: 336000 rows, testing set: 84000 rows

Coefficient vector w = (X^T X)^-1 X^T y:
  w0 (bias)  =  6.794491e+01
  w1 (T)     =  3.720714e-02
  w2 (P)     = -4.219892e-03
  w3 (TC)    = -5.990956e+00
  w4 (SV)    = -5.908089e-02
Equation: Idx = 67.944911 + 0.037207 T - 0.004220 P - 5.990956 TC - 0.059081 SV

Training RMSE = 0.136523
Testing RMSE  = 0.135499
(Standard deviation of Idx = 0.251821; R^2 on test = 0.7086)
Train and test RMSE are almost equal, so the linear model is not overfitting;
the remaining error comes from the relationship not being fully linear.
```

Figure 3. Terminal output for step 6 (normal equation on GasProperties.csv).

Part 2: Supervised Learning for Classification (mlp_classifier.py)

The Iris dataset (150 samples, 4 features, 3 classes) is loaded from sklearn datasets and split 80%/20 % with stratification, yielding 120 training and 30 testing samples. Accuracy is reported for both sets; error = 100% − accuracy.

(3) Baseline MLP with one hidden layer of 3 neurons

Hyperparameters: activation = logistic, solver = SGD, learning_rate_init = 0.001, momentum = 0.9, batch_size = 32, max_iter = 200.

- **Training accuracy: 66.67%** (training error 33.33%)
- **Testing accuracy: 66.67%** (testing error 33.33%)

(4) Tuning hyperparameters (architecture kept at 1 × 3)

We ran a grid search with 5-fold stratified cross-validation over the activation function, solver, learning rate, L2 penalty (alpha), and max_iter. The best setting was activation = logistic, solver = SGD, learning_rate_init = 0.01, alpha = 0.0001 and max_iter = 500 (CV accuracy 97.50%).

- **Training accuracy: 97.50%** (error 2.50%), +30.83 points
- **Testing accuracy: 100.00%** (error 0.00%), +33.33 points

Were we able to improve performance? Yes. With a learning rate of 0.001 and only 200 epochs, the baseline network was under-trained and kept predicting too few classes. Raising the learning rate ten-fold and giving it 500 iterations let the same 3-neuron network converge to a good decision boundary.

```
$ python mlp_classifier.py
Iris dataset: 150 samples, 4 features, 3 classes (setosa, versicolor, virginica)

Training set: 120 samples, testing set: 30 samples

======================================================================
(3) Baseline MLP: 1 hidden layer x 3 neurons
======================================================================
Hyperparameters: activation=logistic, solver=sgd, learning_rate_init=0.001,
                 momentum=0.9, batch_size=32, max_iter=200
Baseline results:
  Training accuracy:  66.67%   (training error 33.33%)
  Testing accuracy:   66.67%   (testing error  33.33%)

======================================================================
(4) Hyperparameter tuning (architecture fixed at 1 x 3 neurons)
======================================================================
Best hyperparameters (5-fold CV accuracy 97.50%):
  activation = logistic
  alpha = 0.0001
  learning_rate_init = 0.01
  max_iter = 500
  solver = sgd
Tuned results:
  Training accuracy:  97.50%   (training error  2.50%)
  Testing accuracy:  100.00%   (testing error   0.00%)
Change vs. baseline: training +30.83 pts, testing +33.33 pts
Explanation: Yes, tuning improved classification performance. The baseline's
small learning rate with logistic units and only 200 SGD epochs left the
network under-trained (stuck predicting too few classes). The tuned
settings above, chiefly a larger learning rate and iteration budget, let
the same 3-neuron network converge to a much better decision boundary.
```

Figure 4. Terminal output for the baseline and the tuned 1 × 3 network.

(5) Increasing network complexity

Keeping the tuned hyperparameters, we tried wider and deeper hidden layers:

Hidden layers	Train accuracy	Test accuracy
(3,)	97.50%	100.00%
(5,)	97.50%	100.00%
(10,)	97.50%	100.00%
(10, 10)	97.50%	100.00%
(20, 10)	98.33%	100.00%

Hidden layers	Train accuracy	Test accuracy
(20, 20, 10)	33.33%	33.33%
(50, 50)	98.33%	100.00%

Were we able to improve performance? Yes, slightly. The (20, 10) network reached 98.33% training accuracy while keeping 100% test accuracy, so it is the better model. Three hidden neurons are a tight bottleneck for separating versicolor from virginica, whose measurements overlap, and more hidden units give the network room to model that boundary. The gain is small because the tuned 1 × 3 network was already close to the ceiling on this easy dataset. More complexity is not automatically better: the three-layer (20, 20, 10) network failed to train with the same plain-SGD settings and predicted a single class (33.33%).

```
$ python mlp_classifier.py   (continued: Part 2.5)
======================================================================
(5) Increasing network complexity (tuned hyperparameters kept)
======================================================================
Hidden layers      Train %    Test %
(3,)                 97.50    100.00
(5,)                 97.50    100.00
(10,)                97.50    100.00
(10, 10)             97.50    100.00
(20, 10)             98.33    100.00
(20, 20, 10)         33.33     33.33
(50, 50)             98.33    100.00

Chosen architecture: (20, 10)
Results:
  Training accuracy:  98.33%   (training error  1.67%)
  Testing accuracy:  100.00%   (testing error   0.00%)
Change vs. tuned 1 x 3 network: training +0.83 pts, testing +0.00 pts
Explanation: Yes. The larger network fits the training data better while
keeping test accuracy at least as high, so it is the better model. Three
hidden neurons are a tight bottleneck for separating versicolor from
virginica, whose measurements overlap; more hidden units give the network
capacity to model that boundary. Gains are small because Iris is easy and
the tuned 1 x 3 network was already near the ceiling. More is not always
better: a deeper network can fail to train at all with the same plain-SGD
settings (see any row stuck at 33.33%, i.e. predicting a single class).
```

Figure 5. Terminal output comparing network sizes.

Part 3: Unsupervised Learning for Clustering (kmeans_clustering.py)

(1) K-means with K = 3 and RMSE per cluster

The program clusters the four Iris features with KMeans(n_clusters = 3); the label column is not used. For each cluster, RMSE = √(mean squared Euclidean distance from each point to its centroid).

Cluster	Size	Centroid [SL, SW, PL, PW]	RMSE
0	62	[5.902, 2.748, 4.394, 1.434]	0.8014
1	50	[5.006, 3.428, 1.462, 0.246]	0.5505
2	38	[6.850, 3.074, 5.742, 2.071]	0.7927
All	150		0.7250

Cluster 1 is the tightest (lowest RMSE); it is the setosa group, which sits far from the other flowers. Clusters 0 and 2 are larger and more spread out because versicolor and virginica overlap.

(2) Cluster quality validation against the real classes and the MLP

Each cluster is mapped to the class that most of its members belong to (0 → versicolor, 1 → setosa, 2 → virginica), and the clusters are compared with the true labels:

Actual \ Cluster	setosa	versicolor	virginica
setosa	50	0	0
versicolor	0	48	2
virginica	0	14	36

K-means agrees with the true classes on 89.33% of all flowers (Adjusted Rand Index 0.730, Normalized Mutual Information 0.758, homogeneity 0.752, completeness 0.765). On the same 20% test split used for the classifier, K-means scores 90.00% while the MLP classifier scores 100.00%.

Discussion. Setosa is recovered perfectly because it is linearly separable from the other two species. All 16 mistakes are between versicolor and virginica, which overlap in feature space; K-means only considers distances to centroids and cannot infer the true boundary, whereas the MLP learns that boundary from the labels during training and is therefore more accurate. Still, without labels, K-means recovers about 89% of the true class structure, so the clusters align well with the actual classes.

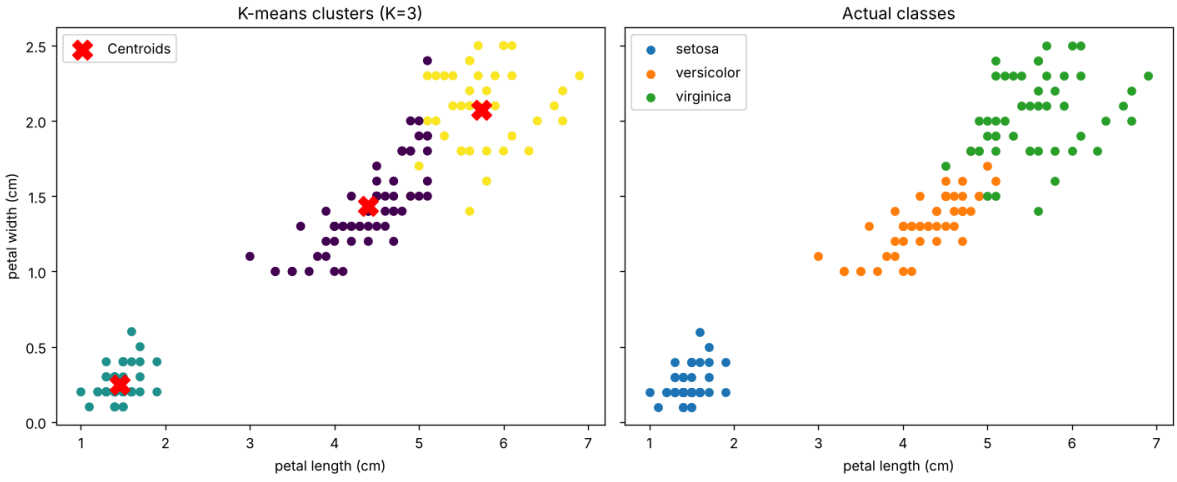


Figure 6. K-means clusters with centroids (left) next to the actual classes (right), petal length vs. petal width.

```
$ python kmeans_clustering.py
Iris dataset: 150 samples, 4 features
Features: sepal length (cm), sepal width (cm), petal length (cm), petal width (cm)

=== (1) K-means clustering (K=3) ===
Cluster 0: size =  62, centroid = [5.902 2.748 4.394 1.434], RMSE = 0.8014
Cluster 1: size =  50, centroid = [5.006 3.428 1.462 0.246], RMSE = 0.5505
Cluster 2: size =  38, centroid = [6.85  3.074 5.742 2.071], RMSE = 0.7927
Overall RMSE (all points to their centroid) = 0.7250
Inertia (sum of squared distances) = 78.8514

=== (2) Cluster quality validation ===
Cluster 0 -> versicolor
Cluster 1 -> setosa
Cluster 2 -> virginica

Confusion matrix (rows = actual class, cols = cluster's class):
                 setosa  versicolor   virginica
     setosa          50           0           0
 versicolor           0          48           2
  virginica           0          14          36

K-means accuracy (clusters mapped to classes) = 89.33%
Adjusted Rand Index   = 0.7302
Normalized Mutual Info = 0.7582
Homogeneity           = 0.7515
Completeness          = 0.7650

K-means vs. MLP classifier (same 20% test set):
Method                            Accuracy
MLP (supervised), train             95.00%
MLP (supervised), test             100.00%
K-means (unsupervised), test        90.00%
K-means (unsupervised), all         89.33%

Discussion:
- One cluster matches Iris setosa exactly; setosa is linearly
  separable from the other two species.
- Versicolor and virginica overlap in feature space, so K-means,
  which only sees distances, mixes some of them. The MLP uses the
  labels during training, so it learns that boundary and scores higher.
- Without any labels, K-means still recovers 89% of the
  true class structure, showing the clusters align well with the classes.

Plot saved to kmeans_clusters.png
```

Figure 7. Terminal output of kmeans_clustering.py.

Part 4.1: Agent Navigation in a Gridworld (gridworld_policy_iteration.py)

The 5 × 5 Gridworld has its start at (4, 0) (red outline), terminal states at (0, 4), (4, 4), and (1, 3) (green outline), an obstacle at (2, 2) (black), and a reward flag at (1, 3) (gold). Every move costs −1, entering a terminal gives +10, and the flag adds +5. With noise, the agent sometimes slips to a different direction than the one it chose. We completed the three missing functions:

(a) Policy evaluation. One sweep updates every non-terminal, non-obstacle state with the Bellman expectation equation $V(s) = \Sigma_a \pi(a|s) \Sigma\square' P(s'|s, a) [R(s, a, s') + \gamma V(s')]$. Terminal and obstacle states keep V = 0.

(b) Policy improvement. For each state, we compute $Q(s, a) = \Sigma\square' P(s'|s, a) [R(s, a, s') + \gamma V(s')]$ for all four actions and make the policy greedy: $\pi(s)$ puts probability 1 on the action with the highest Q

(ties are broken the same way every time). The function returns the new policy and stable = True if it matches the previous policy.

(c) Policy iteration loop. Starting from a random policy, we repeat policy evaluation until the value function stops changing (max $|\Delta V| < 10^{-6}$, at most 100 sweeps), then improve the policy, and stop as soon as the policy is stable.

Results. With $\gamma = 0.95$ and noise = 0, policy iteration converges in 3 iterations. The start state has V = 7.08, and the optimal path goes up the left column and across row 1 to the flag: (4,0) → (3,0) → (2,0) → (1,0) → (1,1) → (1,2) → (1,3). That terminal is worth +15 (10 for the terminal plus 5 for the flag), which is why the agent prefers it over the closer corner terminal at (4, 4). With noise = 0.2, it also converges in 3 iterations, but the start value drops to 5.10, and the agent goes along the bottom row straight to (4, 4) instead. The random slips make the longer route to the flag risky, so the shorter, safer route wins.

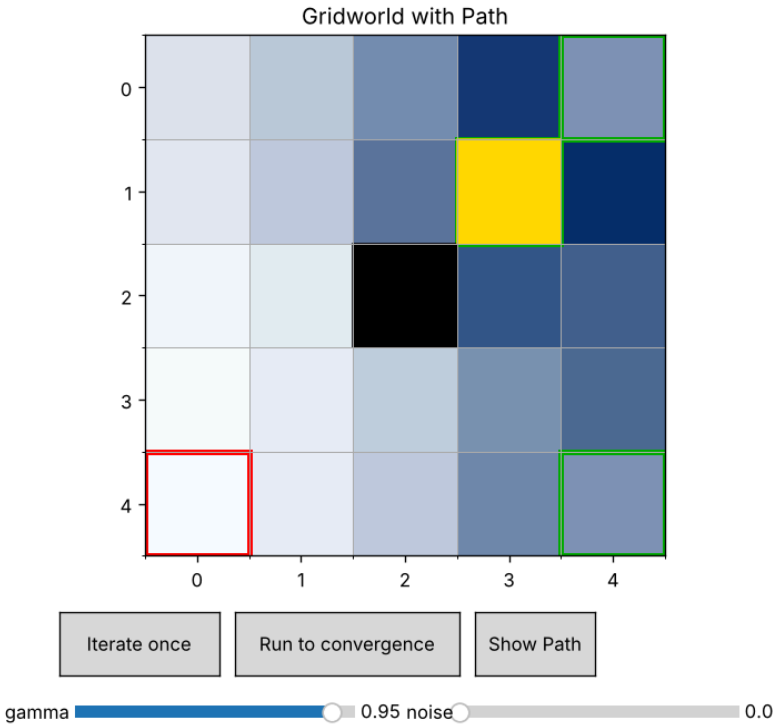


Figure 8. Value-function heatmap after one "Iterate once" from the random policy (darker blue = higher value).

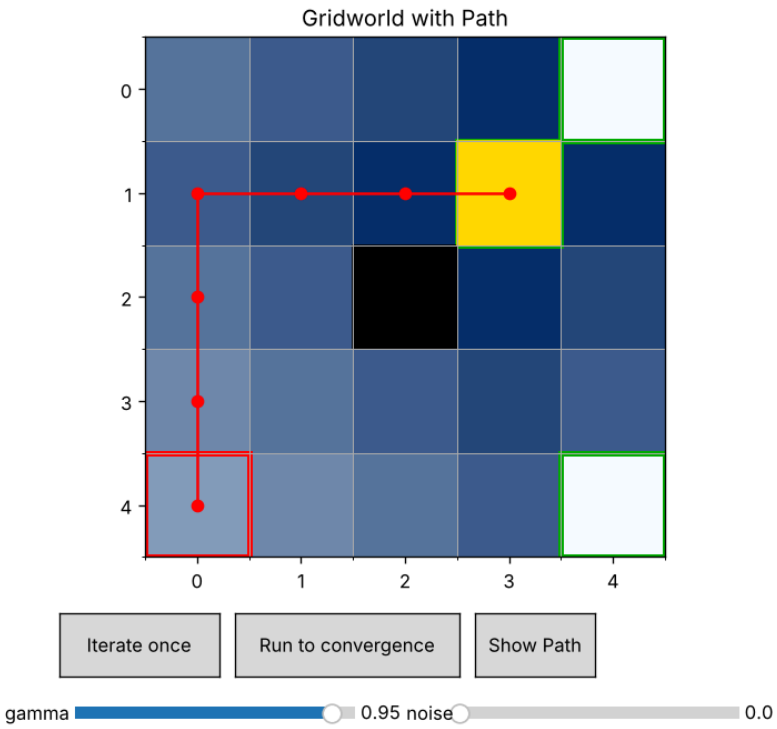


Figure 9. After "Run to convergence" and "Show Path" (γ = 0.95, noise = 0): the learned path from the start to the flag.

```
$ python gridworld_policy_iteration.py   (policy iteration, noise 0.0 and 0.2)
Policy iteration converged after 3 iterations
noise=0.0
V=
 [[10.01 11.59 13.25 15.    0.  ]
 [11.59 13.25 15.    0.   15.  ]
 [10.01 11.59  0.   15.   13.25]
 [ 8.51 10.01 11.59 13.25 11.59]
 [ 7.08  8.51 10.01 11.59  0.  ]]
> > > v T
> > > T <
^ ^ # ^ ^
^ ^ > ^ ^
^ ^ ^ ^ T
path: [(4, 0), (3, 0), (2, 0), (1, 0), (1, 1), (1, 2), (1, 3)]
Policy iteration converged after 3 iterations
noise=0.2
V=
 [[ 8.61 10.38 12.21 14.06  0.  ]
 [ 9.52 11.78 14.32  0.   14.06]
 [ 7.9   9.62  0.   14.32 12.21]
 [ 6.36  7.88  9.65 11.79 10.38]
 [ 5.1   6.6   8.19  9.86  0.  ]]
> > > v T
> > > T <
^ ^ # ^ ^
^ > > ^ ^
> > > > T
path: [(4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
```

Figure 10. Converged values, policy arrows, and paths for noise 0.0 and 0.2 (T = terminal, # = obstacle).

Part 4.2: Snake Agent using a Deep Q-Network (DQN)

(a) Setup and execution

The project is Patrick Loeber's snake-ai-pytorch (https://github.com/patrickloeber/snake-ai-pytorch). It is installed and run with:

git clone https://github.com/patrickloeber/snake-ai-pytorch

pip install torch pygame matplotlib ipython

python agent.py

Running agent.py opens the Pygame window and starts training. The snake is drawn in blue, the food in red, and the current score is shown in the top-left corner.

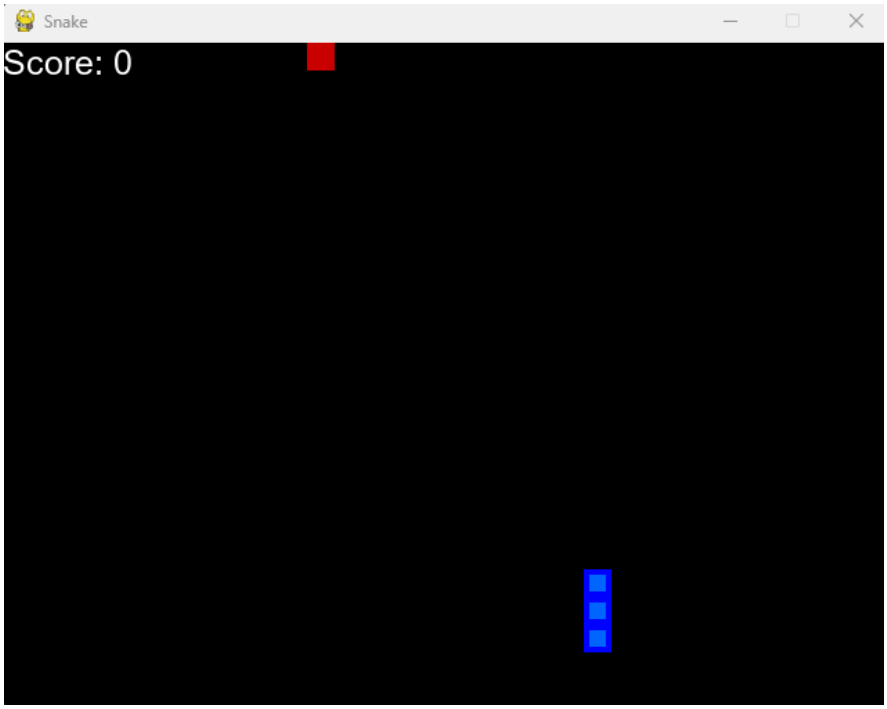


Figure 11. The Snake game running while the DQN agent trains.

After every game, agent.py redraws a live matplotlib plot of the score of each game (blue) and the running mean score (orange). The screenshot was taken early in training, when the agent is still mostly exploring with random moves, so scores are 0 or 1 and the mean drifts around 0.15 to 0.2. As ε drops and the network learns, the scores and the mean score climb.

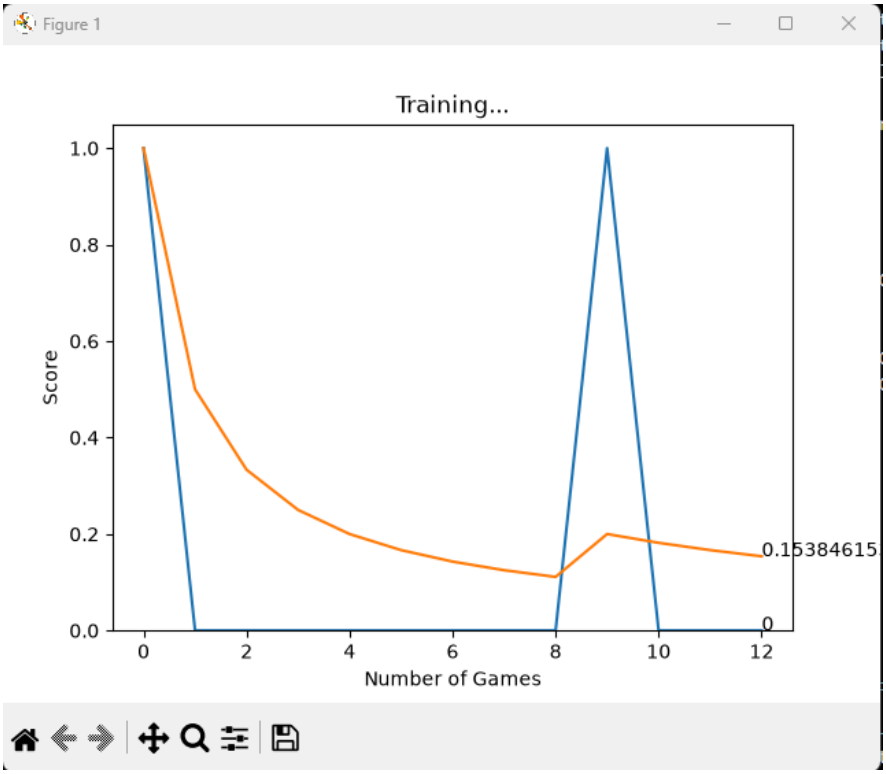


Figure 12. Training plot of the score (blue) and mean score (orange) against the number of games.

The terminal prints one line per finished game with the game number, that game's score, and the best score so far (the record). The Figure(640x480) lines are printed by the plotting helper each time it redraws the chart.

```
Game 14 Score 1 Record: 1
Figure(640x480)
Game 15 Score 1 Record: 1
Figure(640x480)
Game 16 Score 0 Record: 1
Figure(640x480)
Game 17 Score 0 Record: 1
Figure(640x480)
Game 18 Score 1 Record: 1
Figure(640x480)
Game 19 Score 1 Record: 1
Figure(640x480)
```

Figure 13. Terminal output during training.

The answers below are taken from the project's agent.py, game.py, and model.py.

(b) State representation

The state is a vector of 11 boolean values (0 or 1) built in Agent.get_state():

- **Danger (3 values):** whether moving straight, turning right, or turning left would hit a wall or the snake's own body on the next step.
- **Current direction (4 values):** a one-hot encoding of the move direction: left, right, up, or down.
- **Food location (4 values):** whether the food is to the left, right, above, or below the snake's head.

(c) Action space

The agent has **3 possible actions**, relative to its current heading, encoded as one-hot vectors: [1, 0, 0] keep going straight, [0, 1, 0] turn right, [0, 0, 1] turn left. Using relative turns means the snake can never reverse straight into itself.

(d) Reward function

- **Eating food: +10**
- **Dying: −10** (hitting a wall or its own body, or the game timing out when the snake goes 100 × its length frames without eating)
- **Any other move: 0**

(e) DQN output

The network (Linear_QNet: 11 inputs → 256 hidden ReLU units → 3 outputs) produces one value per action. Each output is the estimated Q-value Q(s, a) of that action: the expected total discounted future reward (discount $\gamma = 0.9$) if the agent takes that action in the current state and then keeps playing well. The agent picks the action with the largest Q-value (argmax), except early in training when it sometimes picks a random action to explore ($\varepsilon = 80 -$ number of games played). The network is trained toward the target $r + \gamma \cdot \max Q(s', a')$, using both the most recent move and a replay memory of past moves.